

Conception et Architecture du Système PacLab

Rapport Technique

19 décembre 2025

Introduction

PacLab est une plateforme web interactive dédiée à l'étude algorithmique et comportementale du jeu Pac-Man. Le système combine génération procédurale de labyrinthes, résolution de chemins optimaux et simulation d'intelligences artificielles de fantômes. Cette section présente les choix de conception logicielle et l'architecture technique du projet, en mettant l'accent sur les designs pattern de conception utilisés et l'organisation modulaire du système.

1 Conception Logicielle

1.1 Patrons de Conception Appliqués

La conception du système PacLab repose sur quatre design pattern fondamentaux qui garantissent la maintenabilité, l'extensibilité et la réactivité du système.

1.1.1 Patron Observateur (Observer Pattern)

Le design pattern Observateur constitue le pilier de la communication événementielle dans le système PacLab. Ce design pattern permet à un objet, appelé sujet, de notifier automatiquement un ensemble d'observateurs lors d'un changement d'état.

L'implémentation principale de ce design pattern se trouve dans le moteur de jeu côté client, orchestrant la boucle de jeu et les interactions utilisateur. La classe `GameEngine` agit comme sujet central, elle maintient l'état du jeu et notifie les composants intéressés "Subscribers" lors des changements. La classe `InputHandler` fonctionne comme observateur des événements clavier, elle écoute les touches directionnelles et les convertissent en actions de jeu.

Lorsque l'utilisateur appuie sur une touche directionnelle, l'`InputHandler` enregistre un écouteur d'événements `keydown`. Chaque pression de touche déclenche automatiquement le callback `onDirectionChange` passé au constructeur, qui met à jour la direction du Pacman dans le `GameEngine`. Ce découplage permet à l'`InputHandler` de gérer les entrées clavier sans connaître les détails d'implémentation du moteur de jeu. Le système supporte également une file d'attente de direction, permettant d'anticiper les virages serrés avec la méthode `getQueuedDirection`.

La boucle de jeu "GameLoop" elle-même illustre parfaitement le design pattern Observateur avec `requestAnimationFrame`. Cette API native du navigateur enregistre un callback qui sera automatiquement invoqué avant le prochain rafraîchissement d'écran. Le `GameEngine` s'abonne ainsi implicitement aux événements de rendu du navigateur. À chaque itération, le moteur vérifie l'état du jeu, traite les mouvements, met à jour la grille en supprimant les pastilles collectées, et enregistre la trajectoire dans un tableau pour la sauvegarder dans la base de données afin de l'utiliser après pour la partie simulation.

1.1.2 Patron Singleton (Singleton Pattern)

Le service `PythonBridge` utilise le patron Singleton pour garantir une instance unique de communication avec Python dans toute l'application. L'exportation directe d'une instance instanciée via `module.exports` assure qu'un seul objet `PythonBridge` existe, évitant la duplication de configuration et centralisant la gestion des processus Python.

1.1.3 Patron Fabrique (Factory Pattern)

Le patron Fabrique est utilisé pour la sélection dynamique d'algorithmes de génération de labyrinthes. Le système Python maintient un dictionnaire associant les noms d'algorithmes aux instances de générateurs, permettant l'ajout de nouveaux algorithmes sans modification du code de base. Les deux algorithmes implémentés sont Kruskal avec structure Union-Find, et Prim basé sur une frontière.

1.1.4 Patron Stratégie (Strategy Pattern)

Le patron Stratégie encapsule les quatre personnalités de fantômes dans des classes qui partageant une interface commune. Blinky, Pinky, Inky et Clyde implémentent chacun une méthode de calcul de cible spécifique, permettant au moteur de simulation d'utiliser différents comportements sans avoir besoin de savoir comment ils sont programmés à l'intérieur.

1.2 Architecture Modulaire

1.2.1 Module de Génération de Labyrinthes

Le module de génération transforme un graphe en labyrinthe parfait puis applique optionnellement des imperfections contrôlées. Les algorithmes de génération produisent des labyrinthes garantissant l'existence d'un unique chemin entre deux points. L'imperfeteur peut ensuite créer des boucles et des raccourcis en supprimant sélectivement des murs.

Le système intègre également un mécanisme de placement stratégique de pastilles. Trois stratégies sont disponibles : placement maximal sur toutes les cellules accessibles, placement stratégique sur les intersections et couloirs principaux, et placement minimal aux positions clés. Cette modularité permet d'adapter la densité de pastilles selon les objectifs de la partie.

1.2.2 Module de Jeu Interactif

Le module de jeu permet aux utilisateurs de jouer manuellement et d'enregistrer leurs trajectoires. La classe GameEngine gère la boucle du jeu avec requestAnimationFrame pour un rendu fluide à intervalles réguliers. L'InputHandler capture les événements clavier en temps réel et maintient une file d'attente de directions pour gérer les anticipations de virage.

Le moteur enregistre automatiquement chaque mouvement avec précision, position, direction et nombre de pastilles collectées. Cette trajectoire est sauvegardée en base de données pour l'analyser ou rejouer avec simulation de fantômes. Le système gère également les états de pause et calcule le temps de jeu en excluant les périodes de pause. La partie se termine par une victoire lorsque toutes les pastilles sont collectées.

1.2.3 Module d'Intelligence Artificielle

Le module d'IA implémente les quatre personnalités des fantômes du jeu Pac-Man avec un comportement fidèle au jeu original. Blinky cible directement la position actuelle du joueur, créant une menace constante et prévisible. Pinky applique une logique qui vise quatre cases devant la direction du joueur, anticipant ainsi ses mouvements. Inky calcule géométriquement sa cible en fonction d'un vecteur partant de Blinky. Clyde a un

comportement hybride, il poursuit le joueur lorsqu'il est éloigné et fuit vers son coin de dispersion lorsqu'il s'approche à moins de huit cases.

1.2.4 Module de Simulation

Le moteur de simulation rejoue des trajets déjà enregistrés et ajoute dessus les intelligences artificielles des fantômes. À chaque étape de temps, il regarde s'il y a des collisions, calcule les points gagnés avec les pastilles, et crée des mesures détaillées de performance. Le système enregistre le temps d'exécution, le nombre de nœuds visités par les algorithmes de recherche de chemin, et les collisions.

Cette organisation permet de mesurer clairement l'efficacité des différentes stratégies de résolution et de comparer les performances des algorithmes de recherche de chemin. Les résultats sont enregistrés en format JSON pour rendre plus simple l'analyse statistique et la visualisation plus tard.

Conclusion

L'architecture de PacLab utilise des designs Pattern connus et une technologie moderne. Le design pattern Observateur est beaucoup utilisé pour rendre le système rapide et bien séparé en parties indépendantes. Les designs Pattern Singleton, Factory et Strategy permettent de gérer les services à un seul endroit, de changer facilement les algorithmes et de garder un code modulaire.

Cette base technique solide permet d'ajouter plus tard de nouvelles fonctions pédagogiques et d'analyse à la plateforme.